# PropOS 系统架构设计文档

> 版本: v1.2
>
> 日期: 2026-04-06
>
> 对应 **PRD**: v1.7
>
> 范围: Phase 1 全模块

---

## 目录


0. v1.7 对齐说明
1. 整体架构概览
    - 技术选型
    - 平台能力矩阵
2. 后端 Dart 服务目录结构
    - 核心分层原则
3. PostgreSQL 数据库 Schema
    - 3.1 设计要点概要
    - 3.2 枚举类型清单
    - 3.3 索引策略
4. Flutter App 页面路由结构
    - 4.1 技术选型
    - 4.2 路由树定义
    - 4.3 路由守卫（角色路由映射）
5. RBAC 权限矩阵代码实现
    - 5.1 权限枚举定义
    - 5.2 角色-权限矩阵
    - 5.3 RBAC 中间件（Shelf）
    - 5.4 请求上下文注入（JWT 解析中间件）
    - 5.5 路由注册示例（Controller 层）
    - 5.6 健康检查端点
    - 5.7 中间件管道注册顺序
6. 二房东数据行级隔离 Repository 实现
    - 6.1 设计原则
    - 6.2 RequestContext 结构
    - 6.3 SubleaseRepository 行级隔离实现
    - 6.4 审计日志自动记录

6.5 二房东账号生命周期管理
7. 关键数据流说明
7.1 账单自动生成流程
7.2 WALE 双口径实时计算（v1.7 增强）
7.3 KPI 自动打分（线性插值，含反向指标）
7.4 任务可靠性与人工补偿

---

## 0. v1.7 对齐说明

本版架构文档对齐 PRD v1.7，重点吸收以下约束：

1. Phase 1 以 Must 范围交付为优先，Should 和 Could 能力不阻塞首批上线。
2. 财务模型需支持部分收款、一次收多账单、发票状态与收款状态解耦。
3. KPI 在 Phase 1 定位为正式考核模块（含方案配置/自动冻结/排名/申诉/同比环比/Excel 导出），评分结果为独立考核报告，不对接薪酬系统。
4. 二房东门户需补充登录安全、会话失效、审核 SLA 与版本留痕。
5. 定时任务与消息触发需具备重试、失败列表与人工补偿能力。
6. 合同-单元关系改为 M:N（通过 `contract_units` 中间表，含计费面积与单价）。
7. 押金独立建账（ `deposits` + `deposit_transactions` ），不计入 NOI 收入，状态机全程审计。
8. WALE 双口径计算（收入加权 + 面积加权），精确到天。
9. 合同新增税费口径字段（ `tax_inclusive` / `applicable_tax_rate` ），NOI 统一使用不含税口径。
10. 合同提前终止四种类型（ `termination_type` 枚举），终止后未来账单自动取消、WALE 剩余租期归零。
11. 新增水电抄表（ `meter_readings` ）、商铺营业额对账（ `turnover_reports` ）、导入批次追踪（ `import_batches` ）。
12. KPI 新增反向指标方向字段（ `direction` ），线性插值逻辑翻转。
13. 租客信用评级量化（A/B/C 三级），每月自动重算。
14. PIPL 合规：脱敏还原需记录完整审计日志，合同终止后个人信息保留不超过 3 年。
15. 外部门户强制 HTTPS/TLS 1.2+，密码复杂度要求 8 位以上含大小写字母+数字。

> **交付边界**: 架构设计优先保障资产、合同、账单、收款、工单、二房东填报六条核心链路闭环；KPI 正式考核（含排名/申诉/导出）在 Phase 1 Must 范围内交付；二房东 API 自动推送和增强分析作为可延后能力保留扩展点。

---

## 1. 整体架构概览

```
│                          客户端层                  │
│   ┌──────────────────────────┐   ┌──────────────────┐   ┌──────────────┐ │
│   │       Flutter App        │   │  Web 管理后台 │   │ 微信小程序 │ │
│   │ iOS / Android / 桌面端 │   │ (Flutter Web) │   │ (精简报修)  │ │
│   │ (macOS / Windows)        │   │                  │   │              │ │
│   └──────────┬───────────────┘   └────────┬─────────┘   └──────┬───────┘ │
└──────────────┼──────────────────────────┼──────────────────────┼──────────┘
               │                          │                      │
               └──────────────────────────┼──────────────────────┘
                                          ▼ HTTPS / REST + JWT
┌──────────────────────────────────────────────────────────────────────────┐
│                       Dart 后端服务（Shelf）                              │
│                                                                          │
│   ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────────┐         │
│   │   Auth   │   │ Assets   │   │ Contracts │   │   Finance    │         │
│   │  Module  │   │ Module   │   │  Module  │   │   Module     │         │
│   └──────────┘   └──────────┘   └──────────┘   └──────────────┘         │
│                                                                          │
│   ┌──────────┐   ┌──────────┐   ┌──────────────────────────┐            │
│   │ Workorder│   │ SubLease │   │         KPI Module       │            │
│   │  Module  │   │ Module   │   │                          │            │
│   └──────────┘   └──────────┘   └──────────────────────────┘            │
│                                                                          │
│                          ▲                                               │
│                  ┌──────────────────┐                                    │
│                  │   RBAC 中间件    │  Audit 日志中间件                  │
│                  └──────────────────┘                                    │
└──────────────────────────────────────────────────────────────────────────┘
                                          ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                          数据层                                          │
│   ┌──────────────────────┐   ┌──────────────────────────────────┐       │
│   │   PostgreSQL 主库    │   │   文件存储（S3 兼容 / 本地）     │       │
│   │  (业务数据 + 审计日志) │   │  (CAD 平面图、合同 PDF、照片)    │       │
│   └──────────────────────┘   └──────────────────────────────────┘       │
└──────────────────────────────────────────────────────────────────────────┘
```

## 技术选型

| 层次 | 技术 | 说明 |
| --- | --- | --- |
| 后端框架 | Dart + Shelf | HTTP 服务器框架，中间件管道模式 |
| ORM / 查询 | `postgres` 包（原生 SQL） | 避免 ORM 魔法，行级隔离逻辑显式可控 |
| 数据模型 | `freezed` + `json_serializable` | 不可变数据类，自动序列化 |
| 认证 | JWT（`dart_jsonwebtoken`） | Claims 携带 role + sub*landlord*scope |
| 数据库 | PostgreSQL 15+ | 行级安全 + GIN 索引 |
| 文件转换 | `oc2svg` CLI + 后端调度 | .dwg → SVG/PNG 楼层平面图 |
| APP | Flutter 3.x（`go_router`） | iOS / Android / macOS / Windows / Web 五端复用；桌面端不含 QR 扫码与 FCM 推送，其余功能完整 |

## 平台能力矩阵

| 功能 | iOS | Android | macOS | Windows | Flutter Web |
|---|---|---|---|---|---|
| 全部业务页面（Dashboard、资产、租务、财务） | ✅ | ✅ | ✅ | ✅ | ✅ |
| QR 扫码报修（`mobile_scanner`） | ✅ | ✅ | ❌ → 手动填报 | ❌ → 手动填报 | ❌ → 手动填报 |
| FCM 推送通知（`firebase_messaging`） | ✅ | ✅ | ⚠️ 有限 | ❌ | ❌ |
| 应用内通知 + 轮询（桌面/Web 替代方案） | — | — | ✅ | ✅ | ✅ |
| 文件选择 / Excel 导入（`file_picker`） | ✅ | ✅ | ✅ | ✅ | ✅ |
| 相机拍照上传（`image_picker`） | ✅ | ✅ | ✅ | ⚠️ | ⚠️ |
| SVG 楼层热区图 | ✅ | ✅ | ✅ | ✅ | ✅ |

> **扫码替代方案**：桌面端 `QrScanPage` 降级为 `WorkOrderFormPage（手动输入楼栋/楼层/单元号）`。
>
> **通知替代方案**：桌面端 / Web 使用应用内 Badge 角标 + 30 秒轮询 `/api/alerts/unread`，不依赖 FCM。
>
> **平台判断统一封装**：所有 `Platform.isXxx` / `kIsWeb` 判断集中在 `lib/shared/platform_utils.dart`，页面层不散落平台检测代码。

```
// lib/shared/platform_utils.dart
import 'dart:io';
import 'package:flutter/foundation.dart';

class PlatformUtils {
  static bool get isMobile =>
      !kIsWeb && (Platform.isIOS || Platform.isAndroid);

  static bool get isDesktop =>
```

```
      !kIsWeb && (Platform.isMacOS || Platform.isWindows ||
Platform.isLinux);

  static bool get supportsCamera => isMobile;

  static bool get supportsQrScan => isMobile;

  static bool get supportsPushNotification => isMobile;
}
```

---

## 2. 后端 Dart 服务目录结构

```
backend/
├── packages/                          # 独立 Dart package（零外部依赖，纯计算
库）
│   ├── rent_escalation_engine/        # 租金递增计算引擎
│   │   ├── lib/src/
│   │   │   ├── escalation_rule.dart   # RentEscalationRule sealed class（6种
子类型）
│   │   │   └── rent_calculator.dart   # RentCalculator.compute(rules, date)
→ Money
│   │   ├── test/
│   │   │   └── rent_escalation_test.dart  # 覆盖：固定/阶梯/CPI/混合分段
│   │   └── pubspec.yaml               # name: rent_escalation_engine（无业务
依赖）
│   └── kpi_scorer/                    # KPI 线性插值打分引擎
│       ├── lib/src/
│       │   ├── kpi_metric.dart        # KpiMetric（指标定义 + 满分/及格/不及格阈
值）
│       │   └── kpi_scorer.dart        # KpiScorer.score(metric, actual) →
0~100
│       ├── test/
│       │   └── kpi_scorer_test.dart   # 覆盖：满分/及格/零分/边界值
│       └── pubspec.yaml               # name: kpi_scorer（无业务依赖）
│
├── bin/
│   └── server.dart                    # 入口：启动 Shelf HTTP 服务
├── lib/
│   ├── config/
│   │   ├── app_config.dart            # 环境变量读取（DB_URL、JWT_SECRET 等）
│   │   └── database.dart             # PostgreSQL 连接池初始化
│   │
│   ├── jobs/
│   │   ├── job_runner.dart            # 定时任务统一入口
│   │   ├── job_execution_log.dart     # 任务执行日志与失败列表
│   │   └── retry_scheduler.dart       # 失败重试与人工补偿任务调度
│   │
│   ├── core/
│   │   ├── middleware/
```

```
│   │   │   ├── auth_middleware.dart   # JWT 验证，注入 RequestContext
│   │   │   ├── rbac_middleware.dart   # RBAC 权限检查（见第5节）
│   │   │   ├── rate_limit_middleware.dart # 接口限流（令牌桶，默认 60 req/min/
IP）
│   │   │   └── audit_middleware.dart  # 操作审计日志写入
│   │   ├── errors/
│   │   │   ├── app_exception.dart     # 统一异常基类
│   │   │   └── error_handler.dart     # 全局异常 → HTTP 状态码映射
│   │   ├── pagination.dart            # 分页参数解析与响应包装
│   │   └── request_context.dart      # 请求上下文（用户ID、角色、二房东范围）
│   │
│   ├── modules/
│   │   │
│   │   ├── auth/                      # 认证模块
│   │   │   ├── models/
│   │   │   │   ├── user.dart          # @freezed User
│   │   │   │   └── role.dart         # Role 枚举
│   │   │   ├── repositories/
│   │   │   │   └── user_repository.dart
│   │   │   ├── services/
│   │   │   │   ├── auth_service.dart  # 登录、刷新 Token、登录失败锁定
│   │   │   │   └── token_service.dart # JWT 签发与验证
│   │   │   └── controllers/
│   │   │       └── auth_controller.dart
│   │   │
│   │   ├── organization/              # 组织架构模块（v1.7 新增）
│   │   │   ├── models/
│   │   │   │   ├── department.dart    # @freezed Department（三级组织树）
│   │   │   │   └── managed_scope.dart # @freezed ManagedScope（管辖范围）
│   │   │   ├── repositories/
│   │   │   │   ├── department_repository.dart
│   │   │   │   └── managed_scope_repository.dart
│   │   │   ├── services/
│   │   │   │   └── organization_service.dart  # 组织树 CRUD + 管辖范围配置
│   │   │   └── controllers/
│   │   │       └── organization_controller.dart
│   │   │
│   │   ├── assets/                    # 模块1：资产与空间
│   │   │   ├── models/
│   │   │   │   ├── building.dart      # @freezed Building
│   │   │   │   ├── floor.dart        # @freezed Floor（含 SVG 图层路径）
│   │   │   │   ├── unit.dart         # @freezed Unit（含三业态扩展字段联合体）
│   │   │   │   └── renovation_record.dart
│   │   │   ├── repositories/
│   │   │   │   ├── building_repository.dart
│   │   │   │   ├── floor_repository.dart
│   │   │   │   └── unit_repository.dart
│   │   │   ├── services/
│   │   │   │   ├── unit_service.dart  # 业务逻辑（状态计算、色块聚合）
│   │   │   │   ├── cad_import_service.dart  # .dwg → SVG/PNG 转换调度
│   │   │   │   └── unit_import_service.dart # Excel 批量导入 639 套
```

```
│   │   │   └── controllers/
│   │   │       ├── building_controller.dart
│   │   │       ├── floor_controller.dart
│   │   │       └── unit_controller.dart
│   │   │
│   │   ├── contracts/                  # 模块2：租务与合同
│   │   │   ├── models/
│   │   │   │   ├── tenant.dart         # @freezed Tenant（证件号标注加密）
│   │   │   │   ├── contract.dart      # @freezed Contract（含状态机枚举）
│   │   │   │   ├── contract_status.dart  # ContractStatus 枚举
│   │   │   │   ├── deposit.dart       # @freezed Deposit（押金状态机）
│   │   │   │   └── alert.dart         # Alert 预警记录
│   │   │   │   # ↑ RentEscalationRule 已移入 packages/rent_escalation_engine
│   │   │   ├── repositories/
│   │   │   │   ├── tenant_repository.dart
│   │   │   │   ├── contract_repository.dart
│   │   │   │   ├── deposit_repository.dart    # 押金 CRUD + 状态流转
│   │   │   │   └── alert_repository.dart
│   │   │   ├── services/
│   │   │   │   ├── contract_service.dart     # 合同 CRUD + 状态机转换 + 提前终止
│   │   │   │   ├── deposit_service.dart      # 押金收取/冻结/冲抵/退还/转移 + 审
计
│   │   │   │   ├── wale_service.dart         # WALE 双口径计算（收入加权 + 面积
加权）
│   │   │   │   ├── rent_escalation_service.dart  # 持久化递增规则配置，调用
rent_escalation_engine
│   │   │   │   ├── credit_rating_service.dart    # 租户信用评级自动计算（A/B/
C）
│   │   │   │   └── alert_service.dart        # 预警触发调度（定时任务钩子）
│   │   │   └── controllers/
│   │   │       ├── tenant_controller.dart
│   │   │       ├── contract_controller.dart
│   │   │       ├── deposit_controller.dart   # 押金管理 API
│   │   │       └── wale_controller.dart
│   │   │
│   │   ├── finance/                    # 模块3：财务
│   │   │   ├── models/
│   │   │   │   ├── invoice.dart       # @freezed Invoice（账单）
│   │   │   │   ├── payment.dart      # @freezed Payment（收款主记录）
│   │   │   │   ├── payment_allocation.dart # @freezed PaymentAllocation（核
销分配）
│   │   │   │   ├── expense.dart      # @freezed Expense（运营支出）
│   │   │   │   ├── meter_reading.dart # @freezed MeterReading（水电抄表记录）
│   │   │   │   ├── turnover_report.dart # @freezed TurnoverReport（商铺营业额
申报）
│   │   │   │   ├── kpi_scheme.dart   # @freezed KpiScheme（KPI 考核方案配置）
│   │   │   │   ├── kpi_score.dart    # @freezed KpiScore（评分快照，持久化用）
│   │   │   │   └── kpi_appeal.dart   # @freezed KpiAppeal（考核申诉记录）
│   │   │   ├── repositories/
│   │   │   │   ├── invoice_repository.dart
```

```
│   │   │   │   ├── payment_repository.dart
│   │   │   │   ├── payment_allocation_repository.dart
│   │   │   │   ├── expense_repository.dart
│   │   │   │   ├── meter_reading_repository.dart
│   │   │   │   ├── turnover_report_repository.dart
│   │   │   │   ├── kpi_repository.dart
│   │   │   │   └── kpi_appeal_repository.dart
│   │   │   ├── services/
│   │   │   │   ├── invoice_service.dart      # 自动账单生成（调用
RentCalculator）
│   │   │   │   ├── receivable_service.dart   # 核销分配、部分收款、跨账单收款
│   │   │   │   ├── noi_service.dart          # NOI 实时计算（EGI - OpEx），统
一不含税口径
│   │   │   │   ├── meter_reading_service.dart # 抄表录入 + 自动生成水电费账单
│   │   │   │   ├── turnover_service.dart     # 营业额申报审核 + 分成账单生成
│   │   │   │   ├── kpi_service.dart          # 数据聚合 + 调用 KpiScorer 打分
（含反向指标处理）
│   │   │   │   ├── kpi_ranking_service.dart  # KPI 排名 + 同比环比趋势（v1.7 新
增）
│   │   │   │   └── kpi_export_service.dart   # KPI 考核结果 Excel 导出（v1.7
新增）
│   │   │   └── controllers/
│   │   │       ├── invoice_controller.dart
│   │   │       ├── payment_controller.dart
│   │   │       ├── noi_controller.dart
│   │   │       ├── meter_reading_controller.dart
│   │   │       ├── turnover_controller.dart
│   │   │       ├── kpi_controller.dart
│   │   │       ├── kpi_appeal_controller.dart   # KPI 申诉提交与审核（v1.7 新
增）
│   │   │       └── kpi_export_controller.dart   # KPI 导出端点（v1.7 新增）
│   │   │
│   │   ├── workorders/                # 模块4：工单
│   │   │   ├── models/
│   │   │   │   ├── work_order.dart    # @freezed WorkOrder（含状态机枚举）
│   │   │   │   ├── work_order_status.dart
│   │   │   │   └── supplier.dart     # @freezed Supplier（供应商）
│   │   │   ├── repositories/
│   │   │   │   ├── work_order_repository.dart
│   │   │   │   └── supplier_repository.dart
│   │   │   ├── services/
│   │   │   │   ├── work_order_service.dart   # 状态机转换 + 成本归口
│   │   │   │   └── push_service.dart         # APNs/FCM 推送封装
│   │   │   └── controllers/
│   │   │       └── work_order_controller.dart
│   │   │
│   │   └── subleases/                 # 模块5：二房东穿透
│   │       ├── models/
│   │       │   ├── sublease.dart       # @freezed SubLease（子租赁）
│   │       │   └── sublease_status.dart  # SubLeaseStatus 枚举
│   │       ├── repositories/
```

```
│   │       │   └── sublease_repository.dart  # 行级隔离实现（见第6节）
│   │       ├── services/
│   │       │   ├── sublease_service.dart     # 审核流、填报提醒、版本留痕
│   │       │   ├── portal_session_service.dart # 外部门户会话失效与单点登录控制
│   │       │   └── sublease_import_service.dart  # Excel 批量上传
│   │       └── controllers/
│   │           └── sublease_controller.dart
│   │
│   ├── router/
│   │   └── app_router.dart            # 统一路由注册（mount 各模块路由）
│   │
│   └── shared/
│       ├── encryption.dart            # AES-256 加解密（证件号、手机号）
│       ├── import_batch_service.dart   # 导入批次追踪（整批回滚/部分导入 + 试导入
模式）
│       ├── task_outbox.dart           # 消息发送结果留痕与失败补偿
│       └── validators.dart            # 通用校验工具（含密码复杂度校验）
│
├── test/
│   ├── unit/
│   │   ├── wale_service_test.dart     # WALE 计算单元测试（调用 RentCalculator
mock）
│   │   └── noi_service_test.dart      # NOI 计算单元测试
│   │   # ↑ 递增规则/KPI 打分的纯函数测试已移入各自 package 的 test/ 目录
│   └── integration/
│       └── contract_lifecycle_test.dart
│
├── migrations/                        # 数据库迁移脚本（按版本顺序执行）
│   ├── 001_create_users.sql
│   ├── 002_create_assets.sql
│   ├── 003_create_contracts.sql
│   ├── 004_create_finance.sql
│   ├── 005_create_workorders.sql
│   ├── 006_create_subleases.sql
│   ├── 007_create_deposits.sql           # v1.7: 押金独立建账
│   ├── 008_create_meter_readings.sql     # v1.7: 水电抄表
│   ├── 009_create_turnover_reports.sql   # v1.7: 营业额对账
│   └── 010_create_import_batches.sql     # v1.7: 导入批次追踪
│
└── pubspec.yaml                       # path 依赖本地两个 package
```

`pubspec.yaml` 本地依赖声明：

```
dependencies:
  rent_escalation_engine:
    path: ./packages/rent_escalation_engine
  kpi_scorer:
    path: ./packages/kpi_scorer
```

### 核心分层原则

| 层 | 职责 | 禁止 |
| --- | --- | --- |
| `packages/` | 零副作用纯计算库，无 IO、无 DB、无 HTTP | 不依赖业务模块 |
| `controllers/` | HTTP 请求解析、参数校验、响应序列化 | 不含业务逻辑 |
| `services/` | 业务规则、状态机、数据编排；调用 `packages/` 中的计算函数 | 不含 SQL |
| `repositories/` | 所有 SQL 查询，行级隔离在此层强制 | 不含业务规则 |
| `models/` | `freezed` 不可变数据类；含实体模型与 Command 对象 | 不含副作用 |

**Controller → Service 传递约定**（消除 JSON 与内部模型的歧义）：

- Controller 负责将 HTTP 请求 JSON body 解析为强类型 **Command 对象**（如 `CreateContractCommand`、`UpdateUnitCommand`），再传入 Service；**Service 方法签名只接受强类型参数，禁止接受 `Map<String, dynamic>`**
- 格式校验（字段存在性、类型转换、枚举合法性）在 Controller 层完成，校验失败直接抛 `AppException('INVALID_REQUEST', ..., 400)`
- 业务校验（如合同结束日必须晚于起始日、单元不可重复签约）属于 Service 层，Controller 不做业务判断
- Command 对象定义在所属模块的 `models/` 目录下（与实体模型并列），命名规则：`动词 + 资源名 + Command`，例如 `CreateContractCommand`、`WriteOffInvoiceCommand`
- GET 请求的 Query 参数（分页、筛选）直接以具名参数传入 Service，无需封装 Command 对象

---

## 3. PostgreSQL 数据库 Schema

> 唯一真相源：完整的数据库 Schema 定义（DDL、枚举、索引、约束）统一维护在 `docs/backend/data_model.md`，本节不重复列出 DDL，仅概要说明架构层面的关键设计决策。

## 3.1 设计要点概要

| 关键设计 | 说明 |
| --- | --- |
| 合同-单元 M:N | 通过 `contract_units` 中间表，每条记录独立记录 `billing_area` 与 `unit_price` |
| 押金独立建账 | `deposits` + `deposit_transactions` 双表，状态机 `collected → frozen → partially_credited → refunded`，不计入 NOI |
| 收款核销 | `payments` + `payment_allocations` 双表，支持部分收款、一笔收款核销多张账单 |
| 单元扩展字段 | 三业态差异化属性存入 `units.ext_fields` JSONB，GIN 索引便于过滤 |
| 行级隔离 | 二房东数据通过 `subleases.master_contract_id` 在 Repository 层强制 WHERE 过滤 |
| KPI 快照 | `kpi_score_snapshots` + `kpi_score_snapshot_items` 冗余快照时权重，防止方案修改影响历史 |
| 加密存储 | 证件号、手机号使用 AES-256-GCM 加密，API 层默认脱敏（后4位） |
| 楼层图纸多版本 | 通过 `floor_plans` 表管理多版本图纸，`floors` 表仅存当前生效路径 |
| 数据保留期 | `tenants` / `subleases` 含 `data_retention_until` 字段，合同终止后个人信息保留不超过 3 年（PIPL 合规） |

## 3.2 枚举类型清单

详见 `data_model.md` 第二节，核心枚举包括：

- `property_type`（三业态）、`unit_status`（单元状态）、`contract_status`（合同状态机）
- `escalation_type`（6 种递增类型）、`invoice_status`（账单状态）、`invoice_item_type`（费项）
- `work_order_status`、`work_order_priority`（工单）
- `sublease_occupancy_status`、`sublease_review_status`（含 `draft` 状态）
- `deposit_status`、`termination_type`、`meter_type`、`turnover_approval_status`（v1.7 新增）

### 3.3 索引策略

| 场景 | 关键索引 |
| --- | --- |
| 楼层色块渲染 | `units(floor_id, current_status)` |
| WALE 计算 | `contracts(status, end_date)` covering index |
| 逾期账单催收 | `invoices(status, due_date) WHERE status IN ('issued','overdue')` |
| 二房东数据隔离 | `subleases(master_contract_id)` |
| 工单状态监控 | `work_orders(status, submitted_at DESC)` |
| KPI 快照历史 | `kpi_score_snapshots(evaluated_user_id, period_start)` |
| 审计日志查询 | `audit_logs(resource_type, resource_id)` |
| 单元扩展字段 | `units.ext_fields` GIN 索引（按业态过滤） |

---

## 4. Flutter App 页面路由结构

### 4.1 技术选型

- 路由库：`go_router 14.x`
- 状态管理：`flutter_bloc`
- 鉴权守卫：`GoRouter.redirect` 函数（检查 JWT + 角色权限）
- 导航结构：`ShellRoute`（保持底部栏状态）+ 嵌套路由

### 4.2 路由树定义

```
// lib/router/app_router.dart

final appRouter = GoRouter(
  initialLocation: '/login',
  redirect: _authGuard,   // 全局守卫

  routes: [
    // ── 认证路由 ──
    GoRoute(path: '/login', builder: (ctx, s) => const LoginPage()),
    GoRoute(path: '/forgot-password', builder: (ctx, s) => const
ForgotPasswordPage()),

    // ── 主导航骨架（底部 TabBar） ──
    ShellRoute(
      builder: (ctx, s, child) => MainScaffold(child: child),
```

```
      routes: [

        // ── Tab 1: 概览 Dashboard ──
        GoRoute(
          path: '/dashboard',
          builder: (ctx, s) => const DashboardPage(),    // NOI + 出租率 +
WALE 汇总
          routes: [
            GoRoute(path: 'noi-detail', builder: (ctx, s) => const
NoiDetailPage()),
            GoRoute(path: 'wale-detail', builder: (ctx, s) => const
WaleDetailPage()),
            GoRoute(path: 'kpi',        builder: (ctx, s) => const
KpiDashboardPage()),
            GoRoute(
              path: 'kpi/scheme/:schemeId',
              builder: (ctx, s) => KpiSchemeDetailPage(schemeId:
s.pathParameters['schemeId']!),
            ),
          ],
        ),

        // ── Tab 2: 资产 Assets ──
        GoRoute(
          path: '/assets',
          builder: (ctx, s) => const AssetOverviewPage(),   // 三业态出租率看板
          routes: [
            GoRoute(
              path: 'building/:buildingId',
              builder: (ctx, s) => BuildingDetailPage(id:
s.pathParameters['buildingId']!),
              routes: [
                GoRoute(
                  path: 'floor/:floorId',
                  builder: (ctx, s) => FloorMapPage(           // CAD 热区图
                    buildingId: s.pathParameters['buildingId']!,
                    floorId: s.pathParameters['floorId']!,
                  ),
                  routes: [
                    GoRoute(
                      path: 'unit/:unitId',
                      builder: (ctx, s) => UnitDetailPage(unitId:
s.pathParameters['unitId']!),
                      routes: [
                        GoRoute(path: 'renovation/add',   builder: (ctx, s)
=> RenovationFormPage(unitId: s.pathParameters['unitId']!)),
                        GoRoute(path: 'renovation/:rid',  builder: (ctx, s)
=> RenovationDetailPage(id: s.pathParameters['rid']!)),
                      ],
                    ),
                  ],
                ),
              ],
            ),
```

```
            GoRoute(path: 'import', builder: (ctx, s) => const
UnitImportPage()),   // Excel 批量导入
          ],
        ),

        // ── Tab 3: 租务 Contracts ──
        GoRoute(
          path: '/contracts',
          builder: (ctx, s) => const ContractListPage(),
          routes: [
            GoRoute(path: 'new',        builder: (ctx, s) => const
ContractFormPage()),
            GoRoute(
              path: ':contractId',
              builder: (ctx, s) => ContractDetailPage(id:
s.pathParameters['contractId']!),
              routes: [
                GoRoute(path: 'edit',          builder: (ctx, s) =>
ContractEditPage(id: s.pathParameters['contractId']!)),
                GoRoute(path: 'escalation',    builder: (ctx, s) =>
EscalationConfigPage(contractId: s.pathParameters['contractId']!)),
                GoRoute(path: 'subleases',     builder: (ctx, s) =>
SubleaseListPage(masterContractId: s.pathParameters['contractId']!)),
                GoRoute(path: 'subleases/new', builder: (ctx, s) =>
SubleaseFormPage(masterContractId: s.pathParameters['contractId']!)),
                GoRoute(path: 'renew',         builder: (ctx, s) =>
ContractRenewPage(parentId: s.pathParameters['contractId']!)),
                GoRoute(path: 'terminate',     builder: (ctx, s) =>
ContractTerminatePage(contractId: s.pathParameters['contractId']!)),
                GoRoute(path: 'deposits',      builder: (ctx, s) =>
DepositListPage(contractId: s.pathParameters['contractId']!)),
                GoRoute(path: 'deposits/new',  builder: (ctx, s) =>
DepositFormPage(contractId: s.pathParameters['contractId']!)),
              ],
            ),
          ],
        ),

        GoRoute(
          path: '/tenants',
          builder: (ctx, s) => const TenantListPage(),
          routes: [
            GoRoute(path: 'new',      builder: (ctx, s) => const
TenantFormPage()),
            GoRoute(path: ':tenantId', builder: (ctx, s) =>
TenantDetailPage(id: s.pathParameters['tenantId']!)),
          ],
        ),

        // ── Tab 4: 财务 Finance ──
        GoRoute(
          path: '/finance',
          builder: (ctx, s) => const FinanceOverviewPage(),    // NOI 看板入
口
```

```
          routes: [
            GoRoute(
              path: 'invoices',
              builder: (ctx, s) => const InvoiceListPage(),
              routes: [
                GoRoute(path: ':invoiceId',      builder: (ctx, s) =>
InvoiceDetailPage(id: s.pathParameters['invoiceId']!)),
                GoRoute(path: ':invoiceId/pay',  builder: (ctx, s) =>
PaymentFormPage(invoiceId: s.pathParameters['invoiceId']!)),
              ],
            ),
            GoRoute(path: 'expenses',     builder: (ctx, s) => const
ExpenseListPage()),
            GoRoute(path: 'expenses/new', builder: (ctx, s) => const
ExpenseFormPage()),
            GoRoute(path: 'meter-readings',     builder: (ctx, s) => const
MeterReadingListPage()),
            GoRoute(path: 'meter-readings/new', builder: (ctx, s) => const
MeterReadingFormPage()),
            GoRoute(path: 'turnover-reports',     builder: (ctx, s) => const
TurnoverReportListPage()),
            GoRoute(path: 'turnover-reports/:reportId', builder: (ctx, s) =>
TurnoverReportDetailPage(id: s.pathParameters['reportId']!)),
          ],
        ),

        // ── Tab 5: 工单 Workorders ──
        GoRoute(
          path: '/workorders',
          builder: (ctx, s) => const WorkOrderListPage(),
          routes: [
            GoRoute(path: 'new', builder: (ctx, s) => const
WorkOrderFormPage()),     // 移动端报修入口
            GoRoute(path: 'scan', builder: (ctx, s) => const QrScanPage()),
// 扫码报修
            GoRoute(
              path: ':orderId',
              builder: (ctx, s) => WorkOrderDetailPage(id:
s.pathParameters['orderId']!),
              routes: [
                GoRoute(path: 'approve',  builder: (ctx, s) =>
WorkOrderApprovePage(id: s.pathParameters['orderId']!)),
                GoRoute(path: 'complete', builder: (ctx, s) =>
WorkOrderCompletePage(id: s.pathParameters['orderId']!)),
              ],
            ),
          ],
        ),

      ],
    ),

    // ─── 二房东独立入口（脱离主导航骨架）───
    GoRoute(
```

```
      path: '/sublease-portal',
      builder: (ctx, s) => const SubLandlordPortalPage(),    // 独立简洁界面
      routes: [
        GoRoute(path: 'units',         builder: (ctx, s) => const
SubLandlordUnitListPage()),
        GoRoute(path: 'units/:unitId/fill', builder: (ctx, s) =>
SubleaseFillingPage(unitId: s.pathParameters['unitId']!)),
        GoRoute(path: 'import',        builder: (ctx, s) => const
SubleaseImportPage()),
      ],
    ),

    // ── 系统设置（仅 super_admin / ops_manager）──
    GoRoute(
      path: '/settings',
      builder: (ctx, s) => const SettingsPage(),
      routes: [
        GoRoute(path: 'users',        builder: (ctx, s) => const
UserManagementPage()),
        GoRoute(path: 'users/new',    builder: (ctx, s) => const
UserFormPage()),
        GoRoute(path: 'kpi/schemes',  builder: (ctx, s) => const
KpiSchemeListPage()),
        GoRoute(path: 'kpi/schemes/new', builder: (ctx, s) => const
KpiSchemeFormPage()),
        GoRoute(path: 'escalation/templates', builder: (ctx, s) => const
EscalationTemplateListPage()),
        GoRoute(path: 'alerts',       builder: (ctx, s) => const
AlertCenterPage()),
      ],
    ),
  ],
);
```

### 4.3 路由守卫（角色路由映射）

```
// lib/router/auth_guard.dart

String? _authGuard(BuildContext context, GoRouterState state) {
  final authState = context.read<AuthBloc>().state;

  // 未登录 → 跳登录页
  if (authState is AuthUnauthenticated) {
    if (state.matchedLocation == '/login') return null;
    return '/login';
  }

  final role = authState.user.role;
  final path = state.matchedLocation;

  // 二房东只能访问 sublease-portal 下的路由
  if (role == UserRole.subLandlord && !path.startsWith('/sublease-portal'))
{
```

```
    return '/sublease-portal';
  }

  // 前线员工只能访问工单、只读资产
  if (role == UserRole.frontline) {
    final allowed = ['/dashboard', '/workorders', '/assets'];
    if (!allowed.any((p) => path.startsWith(p))) return '/workorders';
  }

  // 财务只能访问财务模块 + 概览
  if (role == UserRole.financeStaff) {
    final allowed = ['/dashboard', '/finance', '/contracts', '/tenants'];
    if (!allowed.any((p) => path.startsWith(p))) return '/finance';
  }

  return null; // 放行
}
```

---

## 5. RBAC 权限矩阵代码实现

### 5.1 权限枚举定义

```
// lib/core/rbac/permissions.dart

enum Permission {
  // 资产模块
  assetsRead,
  assetsWrite,
  assetsImport,
  cadUpload,

  // 租务模块
  contractsRead,
  contractsWrite,
  tenantsRead,
  tenantsWrite,
  tenantsViewFullCert,   // 查看完整证件号（需二次授权）

  // 财务模块
  financeRead,
  financeWrite,
  invoiceVerify,         // 账单核销
  noiRead,
  meterReadingWrite,     // 水电抄表录入（v1.7）
  turnoverReviewApprove, // 营业额申报审核（v1.7）

  // 押金管理（v1.7）
  depositRead,
  depositWrite,
```

```
  // 工单模块
  workOrderCreate,
  workOrderRead,
  workOrderApprove,
  workOrderComplete,

  // 二房东穿透
  subleaseRead,           // 查看穿透看板
  subleaseWrite,          // 内部人工录入/审核
  subleasePortalAccess,   // 二房东自助录入入口

  // 组织架构（v1.7 新增）
  orgRead,
  orgManage,

  // KPI 申诉（v1.7 新增）
  kpiAppealSubmit,        // 员工提交申诉
  kpiAppealReview,        // 管理者审核申诉
  kpiExport,              // KPI 考核结果导出

  // 系统管理
  userManage,
  kpiSchemeManage,
  kpiSchemeView,
  auditLogRead,
}
```

## 5.2 角色-权限矩阵

```
// lib/core/rbac/role_permissions.dart

const Map<UserRole, Set<Permission>> rolePermissions = {
  UserRole.superAdmin: {
    ...Permission.values,  // 拥有全部权限
  },

  UserRole.opsManager: {
    Permission.assetsRead,
    Permission.assetsWrite,
    Permission.cadUpload,
    Permission.contractsRead,
    Permission.contractsWrite,
    Permission.tenantsRead,
    Permission.tenantsWrite,
    Permission.financeRead,
    Permission.noiRead,
    Permission.depositRead,
    Permission.depositWrite,
    Permission.meterReadingWrite,
    Permission.turnoverReviewApprove,
    Permission.workOrderRead,
    Permission.workOrderApprove,
```

```
  Permission.subleaseRead,
  Permission.subleaseWrite,
  Permission.kpiSchemeManage,
  Permission.kpiSchemeView,
  Permission.kpiAppealReview,
  Permission.kpiExport,
  Permission.orgRead,
  Permission.orgManage,
  Permission.auditLogRead,
},

UserRole.leasingAgent: {
  Permission.assetsRead,
  Permission.contractsRead,
  Permission.contractsWrite,
  Permission.tenantsRead,
  Permission.tenantsWrite,
  Permission.financeRead,     // 只读，用于查看账单
  Permission.depositRead,
  Permission.depositWrite,
  Permission.subleaseRead,
  Permission.subleaseWrite,
  Permission.kpiSchemeView,
  Permission.kpiAppealSubmit,
  Permission.orgRead,
},

UserRole.financeStaff: {
  Permission.assetsRead,
  Permission.contractsRead,
  Permission.tenantsRead,
  Permission.financeRead,
  Permission.financeWrite,
  Permission.invoiceVerify,
  Permission.noiRead,
  Permission.depositRead,
  Permission.depositWrite,
  Permission.meterReadingWrite,
  Permission.turnoverReviewApprove,
  Permission.kpiSchemeView,
  Permission.kpiExport,
  Permission.orgRead,
},

UserRole.frontline: {
  Permission.assetsRead,
  Permission.tenantsRead,     // 只读，不含证件号
  Permission.workOrderCreate,
  Permission.workOrderRead,
  Permission.workOrderComplete,
  Permission.kpiAppealSubmit,
  Permission.orgRead,
},

UserRole.subLandlord: {
```

```
    Permission.subleasePortalAccess,
    // 数据范围由行级隔离控制，不在此处扩展
  },
};
```

### 5.3 RBAC 中间件（Shelf）

```
// lib/core/middleware/rbac_middleware.dart

import 'package:shelf/shelf.dart';
import '../rbac/role_permissions.dart';
import '../request_context.dart';

/// 路由级别权限守卫装饰器
///
/// 使用方式：
///   router.get('/api/contracts', rbacGuard(Permission.contractsRead)
(handler));
Middleware rbacGuard(Permission required) {
  return (Handler innerHandler) {
    return (Request request) async {
      final ctx = RequestContext.of(request);

      if (ctx == null) {
        throw AppException('UNAUTHENTICATED', '未认证', statusCode: 401);
      }

      final perms = rolePermissions[ctx.role] ?? {};
      if (!perms.contains(required)) {
        throw AppException('FORBIDDEN', '无此操作权限', statusCode: 403);
      }

      return innerHandler(request);
    };
  };
}

/// 多权限 OR 检查（拥有其中任一即放行）
Middleware rbacGuardAny(Set<Permission> anyOf) {
  return (Handler innerHandler) {
    return (Request request) async {
      final ctx = RequestContext.of(request);
      if (ctx == null) throw AppException('UNAUTHENTICATED', '未认证',
statusCode: 401);

      final perms = rolePermissions[ctx.role] ?? {};
      if (perms.intersection(anyOf).isEmpty) throw AppException('FORBIDDEN',
'无此操作权限', statusCode: 403);

      return innerHandler(request);
    };
  };
}
```

## 5.4 请求上下文注入（JWT 解析中间件）

```
// lib/core/middleware/auth_middleware.dart

Middleware authMiddleware(TokenService tokenService) {
  return (Handler inner) {
    return (Request request) async {
      // 公开路由白名单（跳过验证）
      const publicPaths = ['/api/auth/login', '/api/auth/refresh'];
      if (publicPaths.any((p) => request.url.path == p)) {
        return inner(request);
      }

      final authHeader = request.headers['authorization'] ?? '';
      if (!authHeader.startsWith('Bearer ')) {
        throw AppException('UNAUTHENTICATED', '缺少有效令牌', statusCode:
401);
      }

      final token = authHeader.substring(7);
      final claims = tokenService.verify(token);
      if (claims == null) {
        throw AppException('UNAUTHENTICATED', '令牌无效或已过期', statusCode:
401);
      }

      // 将上下文注入到请求中（通过 Request.change context）
      final ctx = RequestContext(
        userId: claims['sub'] as String,
        role: UserRole.values.byName(claims['role'] as String),
        boundContractId: claims['bound_contract_id'] as String?, // 二房东专用
      );

      return inner(request.change(context: {RequestContext.key: ctx}));
    };
  };
}
```

## 5.5 路由注册示例（Controller 层）

```
// lib/modules/contracts/controllers/contract_controller.dart

Router contractRoutes(ContractService service) {
  final router = Router();

  router.get('/api/contracts',
      rbacGuard(Permission.contractsRead)(
        (req) async {
          final ctx = RequestContext.of(req)!;
          final page = int.tryParse(req.url.queryParameters['page'] ??
'') ?? 1;
```

```
          final pageSize =
int.tryParse(req.url.queryParameters['pageSize'] ?? '') ?? 20;
          final result = await service.listContracts(ctx, page: page,
pageSize: pageSize);
          return Response.ok(
            jsonEncode({'data': result.items.map((e) =>
e.toJson()).toList(),
                        'meta': {'page': page, 'pageSize': pageSize,
'total': result.total}}),
            headers: {'Content-Type': 'application/json'},
          );
        },
      ));

  router.post('/api/contracts',
      rbacGuard(Permission.contractsWrite)(
        (req) async {
          final ctx = RequestContext.of(req)!;
          final body = jsonDecode(await req.readAsString()) as Map<String,
dynamic>;
          final cmd = CreateContractCommand.fromJson(body); // Controller 层
解析为强类型 Command
          final created = await service.create(cmd, ctx);
          return Response(201,
            body: jsonEncode({'data': created.toJson()}),
            headers: {'Content-Type': 'application/json'},
          );
        },
      ));

  router.get('/api/contracts/<id>',
      rbacGuard(Permission.contractsRead)(
        (req, id) async {
          final result = await service.getById(id, RequestContext.of(req)!);
          return Response.ok(
            jsonEncode({'data': result.toJson()}),
            headers: {'Content-Type': 'application/json'},
          );
        },
      ));

  // 查看完整证件号：需要额外权限
  router.get('/api/tenants/<id>/cert',
      rbacGuard(Permission.tenantsViewFullCert)(
        (req, id) async {
          final result = await service.getFullCert(id,
RequestContext.of(req)!);
          return Response.ok(
            jsonEncode({'data': result}),
            headers: {'Content-Type': 'application/json'},
          );
        },
      ));
```

```
  return router;
}
```

### 5.6 健康检查端点

```
// bin/server.dart 路由注册（无需鉴权，用于 LB / K8s 探针）
router.get('/api/health', (Request req) async {
  // 验证数据库连接可用
  try {
    await db.execute('SELECT 1');
    return Response.ok(jsonEncode({'status': 'ok'}),
        headers: {'Content-Type': 'application/json'});
  } catch (e) {
    return Response.internalServerError(
        body: jsonEncode({'status': 'degraded', 'error': 'db_unreachable'}),
        headers: {'Content-Type': 'application/json'});
  }
});
```

### 5.7 中间件管道注册顺序

```
Request → rate_limit → auth → rbac → audit → Handler → error_handler →
Response
```

- `rate_limit_middleware`：接口级限流，默认 60 req/min/IP（登录接口收紧至 10 req/min）
- `auth_middleware`：JWT 验证，公开路由白名单跳过
- `rbac_middleware`：路由级权限守卫，按 `Permission` 枚举鉴权
- `audit_middleware`：写操作审计日志（合同变更、账单核销、权限变更）
- `error_handler`：全局异常 → 标准 HTTP 错误响应信封

---

## 6. 二房东数据行级隔离 Repository 实现

### 6.1 设计原则

行级隔离在 **Repository 层强制实施**，Service 层通过传递 `RequestContext` 触发，不依赖调用方主动传参，消除遗漏隔离的风险。

| 角色 | 隔离范围 | 实现机制 |
|---|---|---|
| 内部所有角色 | 无数据范围限制 | SQL 无附加 WHERE 条件 |
| `sub_landlord` | 仅限绑定主合同对应的单元和子租赁 | JWT Claims 携带 `bound_contract_id`；Repository 层强制附加 `AND master_contract_id = $scope` |

## 6.2 RequestContext 结构

```
// lib/core/request_context.dart

@freezed
class RequestContext with _$RequestContext {
  const factory RequestContext({
    required String userId,
    required UserRole role,
    String? boundContractId,  // 非 null 当且仅当 role = sub_landlord（对应
users.bound_contract_id）
  }) = _RequestContext;

  /// 是否为二房东角色（需要行级隔离）
  bool get isSubLandlord =>
      role == UserRole.subLandlord && boundContractId != null;

  static const key = 'request_context';
  static RequestContext? of(Request request) =>
      request.context[key] as RequestContext?;
}
```

## 6.3 SubleaseRepository 行级隔离实现

```
// lib/modules/subleases/repositories/sublease_repository.dart

class SubleaseRepository {
  final Connection _db;

  SubleaseRepository(this._db);

  /// 查询子租赁列表
  /// 若 ctx.isSubLandlord == true，自动附加 master_contract_id 过滤
  Future<List<SubLease>> findAll(RequestContext ctx, {
    ReviewStatus? reviewStatus,
    int limit = 50,
    int offset = 0,
  }) async {
    final (scopeClause, params) = _buildScopeClause(ctx);

    var idx = params.length + 1;
    final conditions = [scopeClause];
    if (reviewStatus != null) {
      conditions.add('review_status = \$$idx');
      params.add(reviewStatus.name);
      idx++;
    }

    final where = conditions.join(' AND ');
    final sql = '''
      SELECT s.*, u.unit_no, u.property_type,
             c.contract_no AS master_contract_no
      FROM   subleases s
```

```
      JOIN   units u ON u.id = s.unit_id
      JOIN   contracts c ON c.id = s.master_contract_id
      WHERE  $where
      ORDER  BY s.created_at DESC
      LIMIT  \$$idx OFFSET \$${idx + 1}
    ''';
    params.addAll([limit, offset]);

    final rows = await _db.execute(sql, parameters: params);
    return rows.map(SubLease.fromRow).toList();
  }

  /// 查询单条子租赁（带所有权验证）
  Future<SubLease?> findById(String id, RequestContext ctx) async {
    final (scopeClause, params) = _buildScopeClause(ctx);
    params.add(id);

    final sql = '''
      SELECT * FROM subleases
      WHERE  $scopeClause AND id = \$${params.length}
    ''';
    final rows = await _db.execute(sql, parameters: params);
    return rows.isEmpty ? null : SubLease.fromRow(rows.first);
  }

  /// 新增子租赁（二房东只能在自身主合同范围内的单元创建）
  Future<SubLease> create(CreateSubleaseDto dto, RequestContext ctx) async {
    // 若为二房东，强制 master_contract_id 必须与 JWT 中绑定的合同一致
    if (ctx.isSubLandlord &&
        dto.masterContractId != ctx.boundContractId) {
      throw ForbiddenException('跨主合同操作被拒绝');
    }

    // 验证单元是否在主合同覆盖范围内
    await _assertUnitInScope(dto.unitId, dto.masterContractId);

    const sql = '''
      INSERT INTO subleases
        (master_contract_id, unit_id, sub_tenant_name, sub_tenant_type,
         contact_name, contact_phone_encrypted, contact_phone_hint,
         cert_no_encrypted, cert_no_hint,
         start_date, end_date, monthly_rent,
         occupancy_status, occupant_count, notes,
         review_status, submitted_by)
      VALUES (\$1,\$2,\$3,\$4,\$5,\$6,\$7,\$8,\$9,\$10,\$11,\$12,\$13,\$14,\
$15,
              \$16,\$17)
      RETURNING *
    ''';

    final reviewStatus = ctx.isSubLandlord
        ? ReviewStatus.pendingReview   // 二房东提交后进入待审核
        : ReviewStatus.approved;       // 内部人工录入直接生效
```

```
    final params = [
      dto.masterContractId,
      dto.unitId,
      dto.subTenantName,
      dto.subTenantType.name,
      dto.contactName,
      dto.contactPhoneEncrypted,       // 调用方需提前加密
      dto.contactPhoneHint,
      dto.certNoEncrypted,
      dto.certNoHint,
      dto.startDate.toIso8601String(),
      dto.endDate.toIso8601String(),
      dto.monthlyRent,
      dto.occupancyStatus.name,
      dto.occupantCount,
      dto.notes,
      reviewStatus.name,
      ctx.userId,
    ];

    final rows = await _db.execute(sql, parameters: params);
    return SubLease.fromRow(rows.first);
  }

  /// 更新子租赁（带所有权验证）
  Future<SubLease> update(String id, UpdateSubleaseDto dto, RequestContext
ctx) async {
    // 先查询验证所有权
    final existing = await findById(id, ctx);
    if (existing == null) throw NotFoundException('子租赁记录不存在或无权访问');

    // 二房东只能修改 draft / rejected 状态的记录
    if (ctx.isSubLandlord &&
        existing.reviewStatus != ReviewStatus.draft &&
        existing.reviewStatus != ReviewStatus.rejected) {
      throw ForbiddenException('已提交审核的记录不可修改');
    }

    const sql = '''
      UPDATE subleases SET
        sub_tenant_name = COALESCE(\$2, sub_tenant_name),
        monthly_rent    = COALESCE(\$3, monthly_rent),
        occupancy_status = COALESCE(\$4, occupancy_status),
        notes           = COALESCE(\$5, notes),
        review_status   = \$6,
        updated_at      = NOW()
      WHERE id = \$1
      RETURNING *
    ''';
    final rows = await _db.execute(sql, parameters: [
      id, dto.subTenantName, dto.monthlyRent,
      dto.occupancyStatus?.name, dto.notes,
      ctx.isSubLandlord ? ReviewStatus.draft.name :
ReviewStatus.approved.name,
```

```
    ]);
    return SubLease.fromRow(rows.first);
  }

  // ── 私有辅助方法 ──────────────────────────────────────────

  /// 构建范围 WHERE 子句
  /// - 内部角色：无附加条件（返回恒真条件）
  /// - 二房东：强制附加 master_contract_id = $n
  (String clause, List<Object?> params) _buildScopeClause(RequestContext
ctx) {
    if (ctx.isSubLandlord) {
      return ('master_contract_id = \$1', [ctx.boundContractId]);
    }
    return ('TRUE', []);
  }

  /// 验证 unitId 是否在主合同覆盖范围内
  Future<void> _assertUnitInScope(String unitId, String masterContractId)
async {
    const sql = '''
      SELECT 1 FROM contract_units
      WHERE  contract_id = \$1 AND unit_id = \$2
    ''';
    final rows = await _db.execute(sql, parameters: [masterContractId,
unitId]);
    if (rows.isEmpty) throw BadRequestException('指定单元不在主合同覆盖范围内');
  }
}
```

## 6.4 审计日志自动记录

所有子租赁写操作结束后，Service 层调用公共审计方法：

```
// lib/modules/subleases/services/sublease_service.dart

class SubleaseService {
  final SubleaseRepository _repo;
  final AuditLogRepository _audit;

  Future<SubLease> createSublease(CreateSubleaseDto dto, RequestContext ctx)
async {
    final sublease = await _repo.create(dto, ctx);

    await _audit.log(
      operatorId: ctx.userId,
      module: 'subleases',
      action: 'create',
      entityType: 'sublease',
      entityId: sublease.id,
      newValue: sublease.toJson(),
    );

    return sublease;
```

```
  }
}
```

### 6.5 二房东账号生命周期管理

```
-- 主合同到期后自动冻结二房东账号（数据库触发器）
CREATE OR REPLACE FUNCTION freeze_sub_landlord_on_contract_expiry()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.status IN ('expired', 'terminated') AND OLD.status != NEW.status
THEN
    UPDATE users
       SET is_active = FALSE
     WHERE role = 'sub_landlord'
       AND bound_contract_id = NEW.id;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_freeze_sub_landlord
  AFTER UPDATE OF status ON contracts
  FOR EACH ROW
  EXECUTE FUNCTION freeze_sub_landlord_on_contract_expiry();
```

```
-- 登录失败锁定与会话版本控制由 users 表字段配合 AuthService 实现
-- failed_login_attempts >= 5 时写入 locked_until = NOW() + interval '30
minutes'
-- session_version 每次改密、强制登出或主合同冻结后 +1，旧 JWT 自动失效
```

---

## 7. 关键数据流说明

### 7.1 账单自动生成流程

```
定时任务（每日凌晨1点）
  └ AlertService.checkExpiringContracts()
        └ 查询 end_date ≤ NOW() + 90d 的合同
              └ 写入 alerts（expiry_90d / expiry_60d / expiry_30d）

定时任务（每账期前 N 天）
  └ InvoiceService.generatePeriodInvoices()
        └ 遍历所有 active 合同
              └ RentEscalationService.computeRentForPeriod(contract, period)
                    └ 按阶段规则计算应收租金
              └ 写入 invoices（rent + mgmt_fee + utility）
      └ 写入 invoice_items（按费项拆分）

收款入账
```

```
  └─ PaymentController.createReceipt()
    └─ ReceivableService.allocate(payment, targetInvoices)
      ├─ 默认按"先到期先核销"生成 payment_allocations
      ├─ 支持单笔收款分配到多张 invoice
      └─ 回写 invoice.paid_amount / outstanding_amount / status

水电抄表录入（v1.7 新增）
  └─ MeterReadingController.create()
    └─ MeterReadingService.recordAndBill(reading)
      ├─ 校验 current_reading > previous_reading
      ├─ 计算用量与费用（支持阶梯计价）
      └─ 自动生成水电费账单（invoice_type = 'utility'）

营业额申报审核（v1.7 新增）
  └─ TurnoverController.submitReport()
    └─ TurnoverService.processReport(report)
      ├─ 商户提交 → 财务审核（pending → approved/rejected）
      ├─ 审核通过后自动生成分成账单 = MAX(revenue × rate - base_rent, 0)
      └─ 支持补报/修正（差额账单自动生成）
```

## 7.2 WALE 双口径实时计算（v1.7 增强）

> **v1.7 变更**：WALE 由单一收入加权改为收入加权 + 面积加权双口径；剩余租期精确到天（单位：年）；多单元合同按 `contract_units` 拆分后分别计入，避免重复加权。已终止合同（`termination_type IS NOT NULL`）剩余租期归零，不参与 WALE 计算。

```
GET /api/contracts/wale?groupBy=property_type

ContractController
  └─ WaleService.compute(groupBy: 'property_type', ctx)
       └─ 收入加权 WALE SQL:
            SELECT
              cu.unit_id,
              c.property_type,
              GREATEST(EXTRACT(EPOCH FROM (c.end_date - NOW())) / 86400.0 /
365.25, 0)
                AS remaining_years,
              cu.billing_area * cu.unit_price * 12
                AS annualized_rent,
              cu.billing_area
            FROM contracts c
            JOIN contract_units cu ON cu.contract_id = c.id
            WHERE c.status IN ('active', 'expiring_soon')
              AND c.termination_type IS NULL

       └─ WALE_收入 = Σ(remaining_years × annualized_rent) /
Σ(annualized_rent)
       └─ WALE_面积 = Σ(remaining_years × billing_area) / Σ(billing_area)

       └─ 展示维度：组合级 / 楼栋级 / 业态级 + WALE 趋势图 + 到期瀑布图
```

## 7.3 KPI 自动打分（线性插值，含反向指标）

> **Phase 1 约束**: 本流程输出的是试运行评分结果，快照生成后默认冻结；补录或口径调整需显式触发重算，并保留审计日志。
>
> **v1.7 增强**: 新增 `direction` 字段区分正向/反向指标（ `positive` / `negative` ）。反向指标（K03 租户集中度、K05 工单响应时效、K06 空置周转天数、K08 逾期率）线性插值逻辑翻转：数值越低得分越高。

```
double _interpolateScore(double actual, KpiMetric metric) {
  final fullThreshold = metric.fullScoreThreshold;
  final passThreshold = metric.passThreshold;

  if (metric.direction == 'negative') {
    // 反向指标：数值越低越好
    if (actual <= fullThreshold) return 100.0;
    if (actual >= passThreshold) {
      // 超过及格线：按比例递减 0~60
      final failThreshold = metric.failThreshold;
      return (1 - (actual - passThreshold) / (failThreshold -
passThreshold))
          .clamp(0, 1) * 60;
    }
    // 满分线~及格线之间：线性插值 60~100
    return 100 - (actual - fullThreshold) /
        (passThreshold - fullThreshold) * 40;
  }

  // 正向指标：数值越高越好
  if (actual >= fullThreshold) return 100.0;
  if (actual < passThreshold) {
    // 低于及格线：按比例 0~60
    return (actual / passThreshold * 60).clamp(0, 60);
  }
  // 及格线~满分线：线性插值 60~100
  return 60 + (actual - passThreshold) /
      (fullThreshold - passThreshold) * 40;
}
```

## 7.4 任务可靠性与人工补偿

```
JobRunner.execute(job)
  ├─ 写入 job_execution_log(status=running)
  ├─ 调用业务任务（预警、催收、提醒、导入后处理）
  ├─ 成功 → 标记 success
  └─ 失败 → 记录 error_message / retry_count
            ├─ RetryScheduler 按退避策略自动重试
            └─ 后台失败任务列表支持人工重跑或补发
```

文档结束